Subscribing to Topics

1. Experiment Objective

In this experiment, you will learn how to create a ROS2 topic subscriber on the MicroROS-V2 control board using the micro-ROS framework. Through this example, you will master the subscriber initialization method of micro-ROS, the writing of subscription callback functions, and the use of the executor. This example subscribes to `Int32` messages on the topic `/subscriber` and prints the received data to the serial port.

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
micro-ROS Agent	A proxy program running on the ROS2 host side, responsible for bridging communication between the ESP32 firmware and the ROS2 system, connecting with the microcontroller via the UDP protocol
subscriber	The subscriber, the message-receiving end in ROS2 communication, subscribes to messages on a specified topic and triggers the callback function to process data when a new message arrives
rclc	A ROS2 client library for microcontrollers, providing lightweight node, publisher, subscriber and other APIs, adapted to resource-constrained embedded devices
callback (ON_NEW_DATA)	The subscription callback trigger mode, which invokes the callback function only when a new message is received, avoiding wasted CPU resources from empty polling
QoS (Best Effort)	The "best effort" mode in the quality-of-service policy, which does not guarantee message delivery and ordering, suitable for scenarios with high real-time requirements but tolerant of a small amount of packet loss
executor	The executor, an event-driven scheduler in micro-ROS, responsible for polling subscriber handles and invoking the corresponding callback functions when messages are received

2. Technical Architecture

This experiment builds a one-way data channel from a ROS2 host to an ESP32 firmware subscriber:

- **Hardware layer**: The MicroROS-V2 control board (ESP32) connects to the local network via Wi-Fi
- **Transport layer**: Uses the UDP protocol, communicating with the ROS2 host through the micro-ROS Agent
- **Node configuration**: Node name `esp32_subscriber`, subscribing to topic `/subscriber`
- **Message type**: `std_msgs/msg/Int32`, carrying a 32-bit signed integer
- **QoS mode**: Best Effort, adapting to the unreliable characteristics of the Wi-Fi UDP link

- **Data flow**: ROS2 host publishes a message → Agent → UDP → micro-ROS stack → executor polling → subscription callback → serial port printing

3. Core Topic Quick Reference

Firmware Downlink (Control Commands)

Topic	Message Type	QoS	Field	Range	Direction
`/subscriber`	`std_msgs/msg/Int32`	Best Effort	`data` (int32)	Any integer value	Input, printed to serial port

Usage Example:

```
ros2 topic pub /subscriber std_msgs/msg/Int32 "data: 42"
```

4. Middleware Configuration

micro-ROS uses a **static memory model** (`RCUTILS_AVOID_DYNAMIC_ALLOCATION=ON`), all entity memory is pre-allocated at compile time, and runtime dynamic creation is not supported.

> Configuration file: `~/esp/extra_components/micro_ros_espidf_component/colcon.meta`

`RMW_UXRCE_TRANSPORT`	`udp`	UDP transport (Wi-Fi communication)
`RMW_UXRCE_MAX_PUBLISHERS`	`6`	Maximum number of publishers
`RMW_UXRCE_MAX_SUBSCRIPTIONS`	`6`	Maximum number of subscribers
`RMW_UXRCE_MAX_NODES`	`1`	Maximum number of nodes
`RMW_UXRCE_MAX_SERVICES`	`0`	Services not supported
`RMW_UXRCE_MAX_CLIENTS`	`0`	Clients not supported
`RMW_UXRCE_MAX_HISTORY`	`1`	History depth = 1, no message queuing
`UCLIENT_UDP_TRANSPORT_MTU`	`4096`	Maximum UDP packet size 4 KB
`UCLIENT_MIN_HEARTBEAT_TIME_INTERVAL`	`1`	Agent heartbeat interval 1 ms

Key Constraints:

- `MAX_HISTORY=1` means there is no message buffer queue, so uplink topics (firmware → host) uniformly use **best_effort** QoS
- `MAX_SERVICES=0, MAX_CLIENTS=0` means only topic communication is supported, not Service/Client
- After modifying `colcon.meta`, the micro-ROS library must be recompiled for the changes to take effect: run `idf.py clean-microros` in the project directory to clean and rebuild the micro-ROS library, then `idf.py build` to compile the firmware

3. Experiment Steps

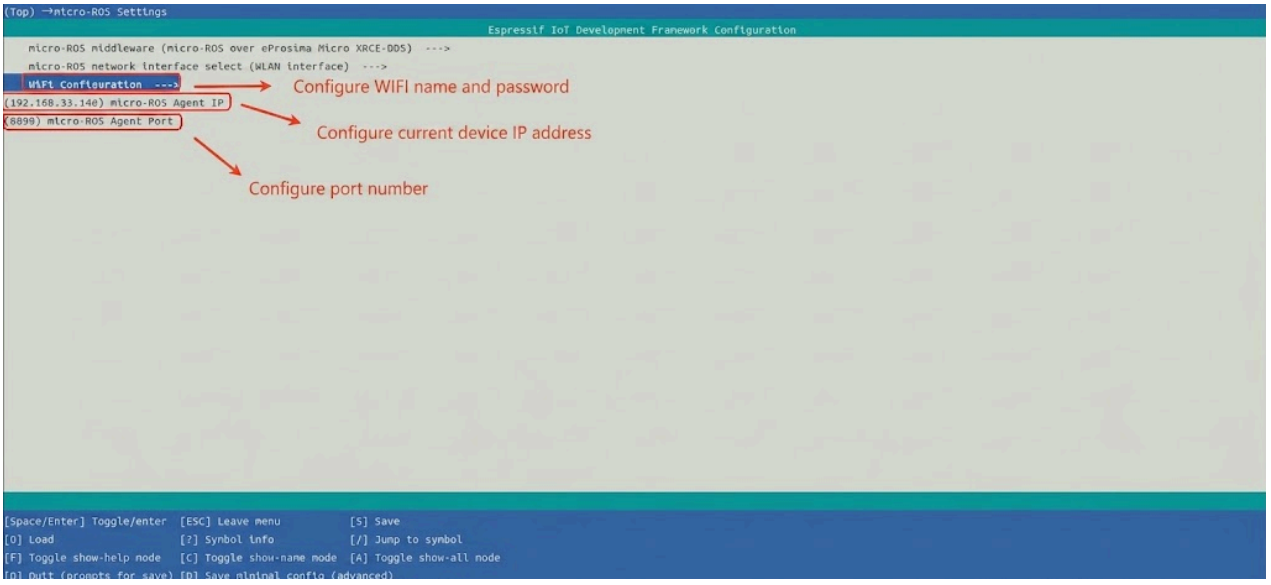
1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✅ Required
USB Type-C cable	✅ Required
Wi-Fi network (2.4 GHz)	✅ Required

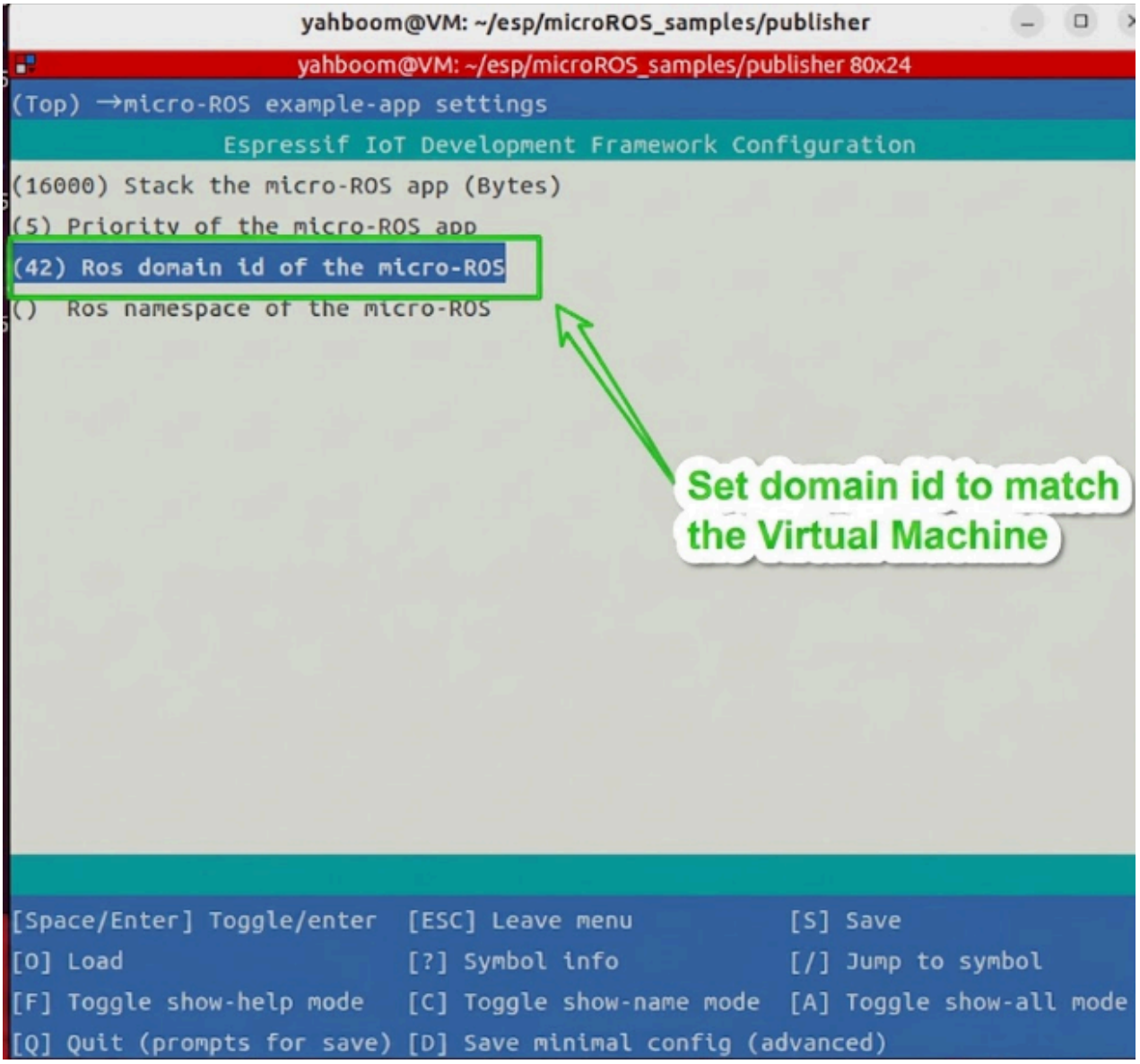
2. Prerequisites

1. The ESP32-IDF development environment has been set up
2. The MicroROS-V2 control board is connected to the computer via USB
3. Configure the Wi-Fi SSID, password, Agent IP and port via `idf.py menuconfig` (the `micro_ros` menu item)

```
cd ~/esp/microROS_samples/subscriber
source ~/esp/esp-idf/export.sh
idf.py menuconfig
```



4. Select example-app setting to set the domain ID to be consistent with the virtual machine



After configuration, press "s" to save and press "q" to exit the configuration interface.

3. Start micro_ros_agent (Terminal 1)

Start the micro-ROS Agent on the ROS2 host:

```
cd ~/uros_ws
source install/local_setup.bash
ros2 run micro_ros_agent micro_ros_agent udp4 --port 8090 -v4
```

4. Build and Flash (Terminal 2)

```
cd ~/esp/microROS_samples/subscriber
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

5. How to Stop

1. Press `Ctrl+]` in Terminal 2 to exit the serial monitor (stop firmware execution)
2. Press `Ctrl+C` in Terminal 1 to stop micro_ros_agent

4. Experiment Result

After the build and flash succeed, publish a message on the ROS2 host with the following command:

```
ros2 topic pub /subscriber std_msgs/msg/Int32 "data: 42"
```

The ESP32 control board serial port will output:

```
I (xxx) MAIN: Agent reachable: 192.168.x.x:8888
I (xxx) MAIN: free heap before support init: xxx
I (xxx) MAIN: stack watermark before support init: xxx
I (xxx) MAIN: free heap after support init: xxx
I (xxx) MAIN: stack watermark after support init: xxx
I (xxx) MAIN: micro-ROS subscriber init done
Received: 42
```

Each time a new message is published on the ROS2 host, the control board serial port will print the received data.

5. Program Analysis

> Source code paths:
>
> - Firmware source code: `~/esp/microROS_samples/subscriber/main/main.c`

1. Main Program Logic

`main/main.c` is the program entry point, and the flow is as follows:

1. `app_main()` calls `uros_network_interface_initialize()` to initialize the Wi-Fi network and disables Wi-Fi power saving mode
2. Creates the `micro_ros_task` task, in which:
    - Configures the UDP address of the micro-ROS Agent and the ROS Domain ID
    - Waits for the Agent to be reachable via `rmw_uros_ping_agent_options()`
    - Initializes the rclc support layer
    - Creates the ROS2 node `esp32_subscriber`
    - Initializes the subscriber in Best Effort mode, with message type `std_msgs/msg/Int32` and topic name `subscriber`
    - Creates an executor and adds the subscription to it
    - Calls `rclc_executor_spin_some()` in the main loop to drive the executor

2. Key Components

Subscriber Configuration:

> Parameter definition file: `~/esp/microROS_samples/subscriber/main/main.c`

Configuration Item	Value	Description
Node name	`esp32_subscriber`	ROS2 node name
Topic name	`subscriber`	Name of the subscribed topic
Message type	`std_msgs/msg/Int32`	32-bit integer message
QoS mode	Best Effort	Reduces pressure on the Wi-Fi UDP link
Callback trigger condition	`ON_NEW_DATA`	Triggers the callback only when new data arrives

Subscription callback function `subscription_callback`:

> Source file: `~/esp/microROS_samples/subscriber/main/main.c`

```
static void subscription_callback(const void *msgin)
{
    const std_msgs__msg__Int32 *msg = (const std_msgs__msg__Int32 *)msgin;
    printf("Received: %ld\n", (long)msg->data);
}
```

The callback function is automatically invoked by the executor when the subscriber receives a new message, printing the `data` field of the message to the serial port.

micro-ROS main task `micro_ros_task`:

> Source file: `~/esp/microROS_samples/subscriber/main/main.c`

```
static void micro_ros_task(void *arg)
{
    (void)arg;

    rcl_allocator_t allocator = rcl_get_default_allocator();
    rclc_support_t support;
    memset(&support, 0, sizeof(support));
    rcl_node_t node = rcl_get_zero_initialized_node();
    rclc_executor_t executor;
    memset(&executor, 0, sizeof(executor));

    subscriber = rcl_get_zero_initialized_subscription();
    memset(&msg_sub, 0, sizeof(msg_sub));

    // Configure the Agent UDP address and initialize init_options
    rcl_init_options_t init_options = rcl_get_zero_initialized_init_options();
    RCCHECK(rcl_init_options_init(&init_options, allocator));
    RCCHECK(rcl_init_options_set_domain_id(&init_options, ROS_DOMAIN_ID));
```

```
    rmw_init_options_t *rmw_options = rcl_init_options_get_rmw_init_options(&init_options);
    RCCHECK(rmw_uros_options_set_udp_address(ROS_AGENT_IP, ROS_AGENT_PORT, rmw_options));

    // 1. Ping the Agent
    while (RMW_RET_OK != rmw_uros_ping_agent_options(
                                AGENT_PING_TIMEOUT_MS,
                                AGENT_PING_ATTEMPTS,
                                rmw_options)) {
        ESP_LOGW(TAG, "Waiting agent: %s:%s", ROS_AGENT_IP, ROS_AGENT_PORT);
        vTaskDelay(pdMS_TO_TICKS(AGENT_RETRY_DELAY_MS));
    }
    ESP_LOGI(TAG, "Agent reachable: %s:%s", ROS_AGENT_IP, ROS_AGENT_PORT);

    // 2. Initialize the rclc support layer
    RCCHECK(rclc_support_init_with_options(&support, 0, NULL, &init_options, &allocator));

    // 3. Create the node
    RCCHECK(rclc_node_init_default(&node, "esp32_subscriber", ROS_NAMESPACE, &support));

    // 4. Create the Best Effort subscriber
    RCCHECK(rclc_subscription_init_best_effort(
        &subscriber,
        &node,
        ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Int32),
        "subscriber"));

    // 5. Create an executor and add the subscription to it
    RCCHECK(rclc_executor_init(&executor, &support.context, EXECUTOR_HANDLE_COUNT,
&allocator));
    RCCHECK(rclc_executor_set_timeout(&executor, RCL_MS_TO_NS(EXECUTOR_TIMEOUT_MS)));
    RCCHECK(rclc_executor_add_subscription(
        &executor, &subscriber, &msg_sub, &subscription_callback, ON_NEW_DATA));

    ESP_LOGI(TAG, "micro-ROS subscriber init done");

    // 6. Main loop: drive the executor
    while (1) {
        RCSOFTCHECK(rclc_executor_spin_some(&executor, RCL_MS_TO_NS(EXECUTOR_TIMEOUT_MS)));
        vTaskDelay(pdMS_TO_TICKS(EXECUTOR_LOOP_DELAY_MS));
    }
}
```

Subscriber initialization flow (corresponding to the `// 1.` ~ `// 6.` comments in the code):

1. Ping the Agent — `rmw_uros_ping_agent_options` (timeout 1000 ms, at most 3 attempts)
2. Initialize the support layer — `rclc_support_init_with_options`
3. Create the node — `rclc_node_init_default("esp32_subscriber")`
4. Create the subscriber — `rclc_subscription_init_best_effort` (topic name `subscriber`, message type `std_msgs/msg/Int32`, `ON_NEW_DATA` trigger mode)
5. Create the executor — `rclc_executor_init` (handle count = 1) + `rclc_executor_add_subscription` (timeout 20 ms)

6. Main loop — `rclc_executor_spin_some` drives the executor

Differences from the Publisher (Lesson 1):

Comparison Item	Publisher (14.1)	Subscriber (14.2)
Communication direction	ESP32 → ROS2 host	ROS2 host → ESP32
Core API	`rclc_publisher_init_best_effort`	`rclc_subscription_init_best_effort`
Data driver	Timer callback	Subscription callback (`ON_NEW_DATA`)
Executor handle	Timer	Subscriber

6. Common Problems

Problem	Cause	Solution
Compilation fails	The ESP-IDF environment is not set up correctly	Run the `source ~/esp/esp-idf/export.sh` command to initialize the ESP-IDF environment
Continuously outputs "Waiting agent" after flashing	The Agent is not started or the configuration is wrong	Confirm that the Agent on the ROS2 host is started, and check the Agent IP and port in menuconfig
No messages received after publishing	Topic name mismatch or network failure	Confirm the topic name is `/subscriber` and check whether the Wi-Fi is properly connected
No messages received after the control board restarts	The Agent needs to rediscover the node	Wait a few seconds for micro-ROS to complete node registration; the Agent will automatically discover the new node